

ERP@HUBKILO: GUIDE DU DEVELOPPEUR – STRUCTURE DES MODULES

Version	Du	Modification	Modifié par
1.0	20/04/2020	Version initiale	NKOUMOU Martin

I. INTRODUCTION

Ce document est une traduction des exigences techniques du développeur afin de produire un rendu de travail portable et de simplifier la prise en main par un autre développeur. Cette section du document se focalise sur la structure des modules et principalement sur Odoo dans sa version 15.

II. STRUCTURE DES MODULES

Il s'agit ici de la structure à respecter pour le développement (ou extension) des modules ODOO.

NB : Les sources natives d'ODOO ne doivent pas être modifiées.

A. STRUCTURE PAR DEFAUT

`addons\` (dossier racine du serveur d'applications)

`module\` (dossier du module ajouté)

`__init__.py` (listes de dossiers et/ou fichiers à compiler)

`__manifest__.py` (méta-informations requises pour l'intégration)

`module_menu.xml` (structures des menus à ajouter)

`module_report.xml` (définitions des rapports disponibles)

`data\` (définitions des fichiers XML servant au pré chargement des données)

`models\` (l'ensemble des objets métiers – fichiers .py)

`reports\` (l'ensemble des rapports et parseurs)

`security\` (définitions de groupes, droits et règles)

`static\` (ensemble des fichiers JS et IMG utilisés par le module)

`views\` (design des vues et leurs déclencheurs – actions en XML)

Guide développeur – structuration des modules

wizards\ (*controllers, vues et actions pour les assistants*)

i18n\ (*répertoire des fichiers de langues pour les traductions*)

B. LISTE DES MODULES A DEVELOPPER

Ces noms sont des propositions. Le plus important est de respecter une logique dans la nomenclature et de créer un module pour chaque module métier des spécifications techniques.

Guide développeur – structuration des modules

Module	Nom technique	Responsable
Gestion des entités de base de HUB KILO (voyageurs et destinataires)	m0st_hk_base	TENDA
Gestion des expéditions par voie terrestre	m1st_hk_groundshipping	TENDA
Gestion des expéditions par voie aérienne	m2st_hk_airshipping	TENDA
Gestion des expéditions par voie maritime	m3st_hk_seashipping	TENDA
Gestion des paiements	m4st_hk_payment	MARTIN
Gestion des tableaux de bord, des états et diagrammes d'analyses	m5st_hk_dashboard	MARTIN

III. NOMENCLATURE

Une bonne nomenclature facilite **énormément** l'appropriation des codes source, et donc sa prise en main par tiers pour maintenance, évolution, ... ; bref, **l'utilité pérenne des codes source en dépend**.

Dans le cas où chaque équipe est entièrement responsable de son code, il est important de fixer des bases (standards à respecter) pour préserver une homogénéité en ce qui concerne la nomenclature.

Toute abréviation utilisée doit être intuitive – sa définition doit facilement être déduite de son emplacement dans l'arborescence des fichiers. Ex.

A. ADDONS

Cf. sous-titre II.B. (Liste des modules).

B. OBJET / CLASSE

1. Chaque modèle à un nom complet (fullname), un nom simple (shortname) et un pseudo (nickname)
 - Fullname = <addon>.<shortname>
 - ex. m1st_hk_groundshipping
2. Le fullname est utilisé pour l'attribut **_name** et par conséquent pour les cas suivants :
 - attribut **_inherit**
 - paramètre **relation** pour la définition d'un champ (fields._)
 - tag **<field name="model"/>** pour les **record** xml
 - ...
3. Le shortname est utilisé pour :
 - Nom du fichier
 - Nom de la classe python
 - ... et partout ailleurs où la valeur de l'attribut **_name** n'est pas celle requise
 - Ex. hk_groundshipping

Guide développeur – structuration des modules

4. Le nickname est utilisé comme nom d'un champ avec les suffixe '**_id**', '**_ids**' ou comme paramètre d'une méthode. Le pseudo est généralement une abréviation du shortname ou équivalent au shortname.

- Ex. groundshipping_id

C. XML (VIEWS, ACTIONS, GROUPS, ...)

1. L'id de chaque élément xml est structuré ainsi, **<xml_element>_<fullname>_<context>**.

- Ex.

- *'view_m1st_hk_groundshipping_form'*,
- *'menu_groundshipping_root'*,
- *'action_m1st_hk_groundshipping_wizard'*,
- *'group_groundshipping_manager'*,

IV. MODELES

1. Les entités ci-après doivent être des héritages de res.partner
 - Voyageur
 - Destinataire
 - Expéditeur
2. Lorsqu'un objet métier d'Odoo est utilisé dans un « field », le nom du champ doit obligatoirement être du genre « shortname_id(s) ». Par exemple pour res.partner, le champ Many2one devra être « partner_id » tandis que les O2m et M2m devront être « partner_ids ». si pour la relation intervient plusieurs fois dans l'objet, alors faire précéder le « shortname » du contexte. Par exemple. « voyageur_partner_id & destinataire_partner_id »
3. Toute classe/objet représentant une fonctionnalité métier (contrairement à une configuration ou un utilitaire) doit hériter des objets [**'mail.thread'**, **'mail.activity.mixin'**],
4. La signature de chaque méthode doit être suivi de ; ' ' ' (1) commentaire descriptif de sa fonction, (2) listing / description des paramètres spécifiques, (3) valeur renvoyé ' ' '. De plus les méthodes obéissent à la nomenclature suivante :
 - def myFunctionName(self, param_1, param_2, ..., param_n)

V. STRUCTURATION DES REPERTOIRES DE TRAVAIL

1. Au niveau du poste développeur

Le poste développeur devra avoir deux répertoires de travail que sont :

- **Dev** : Il s'agit du répertoire de développement qui contient les modules
- **Addons** : Il s'agit du répertoire dans lequel les tests sont effectués sur la version locale d'ODOO du poste développeur. C'est un répertoire de validation.